Homework 4

1. Gradients, Jacobians, and Hessians

The *gradient* of a scalar-valued function $g\colon \mathbb{R}^n \to \mathbb{R}$ is the column vector of length n, denoted as ∇g, containing the derivatives of components of g with respect to the input variables:

$$(\nabla g(\vec{x}))_i = \frac{\partial g}{\partial x_i}(\vec{x}),\ i = 1, \ldots n. \tag{1}$$

The *Hessian* of a scalar-valued function $g\colon \mathbb{R}^n \to \mathbb{R}$ is the $n \times n$ matrix, denoted as $\nabla^2 g$, containing the second derivatives of components of g with respect to the input variables:

$$(\nabla^2 g(\vec{x}))_{ij} = \frac{\partial^2 g}{\partial x_i \partial x_j}(\vec{x}),\quad i = 1, \ldots, n,\quad j = 1, \ldots, n. \tag{2}$$

The *Jacobian* of a vector-valued function $\vec{g}\colon \mathbb{R}^n \to \mathbb{R}^m$ is the $m \times n$ matrix, denoted as $D\vec{g}$, containing the derivatives of components of $\vec{g}$ with respect to the input variables:

$$(D\vec{g}(\vec{x}))_{ij} = \frac{\partial g_i}{\partial x_j}(\vec{x}),\quad i = 1, \ldots, m,\quad j = 1, \ldots, n. \tag{3}$$

For the remainder of the class, we will repeatedly have to take gradients, Hessians and Jacobians of functions we are trying to optimize. This exercise serves as a warm up for future problems.

For the first two parts, suppose $A \in \mathbb{R}^{n\times n}$ is a square matrix whose entries are denoted a_{ij} and whose rows are denoted $\vec{a}_1^\top, \ldots, \vec{a}_n^\top$, and $\vec{b} \in \mathbb{R}^n$ is a vector whose entries are denoted b_i.

(a) Compute the Jacobians for the following functions.

i. $\vec{g}(\vec{x}) = A\vec{x}$.

Solution: We compute each partial derivative and reconstitute $D\vec{g}$ at the end. That is,

$$\begin{aligned}
[D\vec{g}(\vec{x})]_{jk} &= \frac{\partial g_j}{\partial x_k}(\vec{x}) && (4)\\
&= \frac{\partial (A\vec{x})_j}{\partial x_k} && (5)\\
&= \frac{\partial (\vec{a}_j^\top \vec{x})}{\partial x_k} && (6)\\
&= (\vec{a}_j)_k && (7)\\
&= a_{jk} && (8)\\
&= [A]_{jk}, && (9)
\end{aligned}$$

where $[A]_{jk}$ is the (j,k)th entry of A. Thus $D\vec{g}(\vec{x}) = A$.

ii. $\vec{g}(\vec{x}) = f(\vec{x})\vec{x}$ where $f : \mathbb{R}^n \mapsto \mathbb{R}$ is differentiable.

Solution: We again compute each partial derivative using the scalar product rule, obtaining

$$[D\vec{g}(\vec{x})]_{jk} = \frac{\partial g_j}{\partial x_k}(\vec{x}) \tag{10}$$

$$= \frac{\partial(f(\vec{x})x_j)}{\partial x_k} \tag{11}$$

$$= f(\vec{x})\frac{\partial x_j}{\partial x_k} + x_j\frac{\partial f}{\partial x_k}(\vec{x}) \tag{12}$$

$$= x_j[\nabla f(\vec{x})]_k + \begin{cases} f(\vec{x}), & \text{if } j = k \\ 0, & \text{if } j \neq k. \end{cases} \tag{13}$$

This gives a Jacobian whose entries are

$$[D\vec{g}(\vec{x})]_{jk} = x_j[\nabla f(\vec{x})]_k, \qquad \forall j \neq k \tag{14}$$

$$[D\vec{g}(\vec{x})]_{jj} = x_j[\nabla f(\vec{x})]_k + f(\vec{x}), \qquad \forall j. \tag{15}$$

For the purpose of self-grades it is fine to stop here. But one can also write the Jacobian as

$$D\vec{g}(\vec{x}) = \vec{x}[\nabla f(\vec{x})]^\top + f(\vec{x})I. \tag{16}$$

iii. $\vec{g}(\vec{x}) = f(A\vec{x} + \vec{b})\vec{x}$ where $f : \mathbb{R}^n \mapsto \mathbb{R}$ is differentiable.

Solution: We compute each partial derivative using the scalar product rule and chain rule, obtaining

$$[D\vec{g}(\vec{x})]_{jk} = \frac{\partial g_j}{\partial x_k}(\vec{x}) \tag{17}$$

$$= \frac{\partial f(A\vec{x} + \vec{b})x_j}{\partial x_k} \tag{18}$$

$$= f(A\vec{x} + \vec{b})\frac{\partial x_j}{\partial x_k} + x_j\frac{\partial f(A\vec{x} + \vec{b})}{\partial x_k} \tag{19}$$

$$= f(A\vec{x} + \vec{b})\frac{\partial x_j}{\partial x_k} + x_j\sum_{\ell=1}^{n}\frac{\partial f(A\vec{x} + \vec{b})}{\partial (A\vec{x} + \vec{b})_\ell} \cdot \frac{\partial (A\vec{x} + \vec{b})_\ell}{\partial x_k} \tag{20}$$

$$= f(A\vec{x} + \vec{b})\frac{\partial x_j}{\partial x_k} + x_j\sum_{\ell=1}^{n}[\nabla f(A\vec{x} + \vec{b})]_\ell \cdot \frac{\partial (\vec{a}_\ell^\top \vec{x} + b_\ell)}{\partial x_k} \tag{21}$$

$$= f(A\vec{x} + \vec{b})\frac{\partial x_j}{\partial x_k} + x_j\sum_{\ell=1}^{n}[\nabla f(A\vec{x} + \vec{b})]_\ell \cdot (\vec{a}_\ell)_k \tag{22}$$

$$= f(A\vec{x} + \vec{b})\frac{\partial x_j}{\partial x_k} + x_j\sum_{\ell=1}^{n}[\nabla f(A\vec{x} + \vec{b})]_\ell \cdot a_{\ell k} \tag{23}$$

$$= f(A\vec{x} + \vec{b})\frac{\partial x_j}{\partial x_k} + x_j\sum_{\ell=1}^{n}[\nabla f(A\vec{x} + \vec{b})]_\ell \cdot [A^\top]_{k\ell} \tag{24}$$

$$= f(A\vec{x} + \vec{b})\frac{\partial x_j}{\partial x_k} + x_j[A^\top \nabla f(A\vec{x} + \vec{b})]_k \tag{25}$$

$$= x_j[A^\top \nabla f(A\vec{x} + \vec{b})]_k + \begin{cases} f(A\vec{x} + \vec{b}), & \text{if } j = k \\ 0 & \text{if } j \neq k. \end{cases} \tag{26}$$

This gives a Jacobian whose entries are

$$[D\vec{g}(\vec{x})]_{jk} = x_j[A^\top \nabla f(A\vec{x} + \vec{b})]_k, \qquad \forall j \neq k \tag{27}$$

$$[D\vec{g}(\vec{x})]_{jj} = x_j[A^\top \nabla f(A\vec{x} + \vec{b})]_k + f(A\vec{x} + \vec{b}), \qquad \forall j. \tag{28}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

For the purpose of self-grades it is fine to stop here. But one can also write the Jacobian as

$$D\vec{g}(\vec{x}) = \vec{x}[A^\top \nabla f(A\vec{x}+\vec{b})]^\top + f(A\vec{x}+\vec{b})I \tag{29}$$
$$= \vec{x}[\nabla f(A\vec{x}+\vec{b})]^\top A + f(A\vec{x}+\vec{b})I. \tag{30}$$

(b) Compute the gradients and Hessians for the following functions.

i. $g_1(\vec{x}) = \vec{x}^\top A\vec{x}$.

Solution: We have

$$g_1(\vec{x}) = \vec{x}^\top A\vec{x} \tag{31}$$
$$= \vec{x}^\top \begin{bmatrix} \vec{a}_1^\top \\ \vdots \\ \vec{a}_n^\top \end{bmatrix} \vec{x} \tag{32}$$
$$= \vec{x}^\top \begin{bmatrix} \vec{a}_1^\top \vec{x} \\ \vdots \\ \vec{a}_n^\top \vec{x} \end{bmatrix} \tag{33}$$
$$= \sum_{i=1}^n x_i(\vec{a}_i^\top \vec{x}). \tag{34}$$

Taking the derivative with respect to any x_j yields

$$\frac{\partial g_1}{\partial x_j}(\vec{x}) = \frac{\partial}{\partial x_j} \sum_{i=1}^n x_i(\vec{a}_i^\top \vec{x}) \tag{35}$$
$$= \frac{\partial}{\partial x_j} \left[x_j(\vec{a}_j^\top \vec{x}) + \sum_{\substack{i=1 \\ i\neq j}}^n x_i(\vec{a}_i^\top \vec{x}) \right] \tag{36}$$
$$= \frac{\partial}{\partial x_j} x_j(\vec{a}_j^\top \vec{x}) + \sum_{\substack{i=1 \\ i\neq j}}^n \frac{\partial}{\partial x_j} x_i(\vec{a}_i^\top \vec{x}) \tag{37}$$
$$= x_j \frac{\partial}{\partial x_j}(\vec{a}_j^\top \vec{x}) + \vec{a}_j^\top \vec{x} + \sum_{\substack{i=1 \\ i\neq j}}^n x_i \frac{\partial}{\partial x_j}(\vec{a}_i^\top \vec{x}) \tag{38}$$
$$= x_j(\vec{a}_j)_j + \vec{a}_j^\top \vec{x} + \sum_{\substack{i=1 \\ i\neq j}}^n x_i(\vec{a}_i)_j \tag{39}$$
$$= \vec{a}_j^\top \vec{x} + \sum_{i=1}^n x_i(\vec{a}_i)_j \tag{40}$$
$$= \sum_{i=1}^n a_{ji}x_i + \sum_{i=1}^n a_{ij}x_i \tag{41}$$
$$= (A\vec{x})_j + (A^\top \vec{x})_j \tag{42}$$
$$= [(A + A^\top)\vec{x}]_j. \tag{43}$$

Thus we have

$$\nabla g_1(\vec{x}) = (A + A^\top)\vec{x}. \tag{44}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

For the Hessian, notice that

$$
\begin{aligned}
[\nabla^2 g_1(\vec{x})]_{jk} &= \frac{\partial^2 g_1}{\partial x_j \, \partial x_k}(\vec{x}) && (45)\\
&= \frac{\partial [\nabla g_1]_j}{\partial x_k}(\vec{x}) && (46)\\
&= \frac{\partial}{\partial x_k}\left[\sum_{i=1}^n a_{ji}x_i + \sum_{i=1}^n a_{ij}x_i\right] && (47)\\
&= \left[\sum_{i=1}^n \frac{\partial}{\partial x_k} a_{ji}x_i + \sum_{i=1}^n \frac{\partial}{\partial x_k} a_{ij}x_i\right] && (48)\\
&= a_{jk} + a_{kj} && (49)\\
&= (A + A^\top)_{jk}. && (50)
\end{aligned}
$$

Thus we have

$$\nabla^2 g_1(\vec{x}) = A + A^\top. \tag{51}$$

Notice the important special case that if A is symmetric then $\nabla g_1(\vec{x}) = 2A\vec{x}$ and $\nabla^2 g_1(\vec{x}) = 2A$.

ii. $g_2(\vec{x}) = \|\vec{x}\|_2^2$.

Solution: We take $A = I$ in g_1, obtaining

$$\nabla g_2(\vec{x}) = 2\vec{x} \qquad \text{and} \qquad \nabla^2 g_2(\vec{x}) = 2I. \tag{52}$$

iii. $g_3(\vec{x}) = g_2(A\vec{x} - \vec{b}) = \|A\vec{x} - \vec{b}\|_2^2$. (Use the chain rule and the Jacobians computed in part 1(a).)

Solution: As required, we use the chain rule: we compute

$$
\begin{aligned}
\nabla g_3(\vec{x}) &= [Dg_3(\vec{x})]^\top && (53)\\
&= ([Dg_2(A\vec{x} - \vec{b})][D(A\vec{x} - \vec{b})])^\top && (54)\\
&= ([\nabla g_2(A\vec{x} - \vec{b})]^\top [D(A\vec{x} - \vec{b})])^\top && (55)\\
&= [D(A\vec{x} - \vec{b})]^\top [\nabla g_2(A\vec{x} - \vec{b})] && (56)\\
&= [A]^\top [2(A\vec{x} - \vec{b})] && (57)\\
&= 2A^\top (A\vec{x} - \vec{b}). && (58)
\end{aligned}
$$

For the Hessian, note that

$$
\begin{aligned}
\nabla^2 g_3(\vec{x}) &= D[\nabla g_3(\vec{x})] && (59)\\
&= D[2A^\top (A\vec{x} - \vec{b})] && (60)\\
&= 2D[A^\top A\vec{x} - A^\top \vec{b}] && (61)\\
&= 2A^\top A. && (62)
\end{aligned}
$$

iv. $g_4(\vec{x}) = \log(\sum_{i=1}^n e^{x_i})$.

Solution: We use component-wise derivatives and the scalar-valued chain rule:

$$\frac{\partial g_4}{\partial x_j}(\vec{x}) = \frac{\partial}{\partial x_j} \log\left(\sum_{i=1}^n e^{x_i}\right) \tag{63}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

$$= \frac{\frac{\partial}{\partial x_j} \sum_{i=1}^n e^{x_i}}{\sum_{i=1}^n e^{x_i}} \tag{64}$$

$$= \frac{\sum_{i=1}^n \frac{\partial}{\partial x_j} e^{x_i}}{\sum_{i=1}^n e^{x_i}} \tag{65}$$

$$= \frac{e^{x_j}}{\sum_{i=1}^n e^{x_i}}. \tag{66}$$

Thus the gradient is of the form

$$\nabla g_4(\vec{x}) = \begin{bmatrix} \partial g_4/\partial x_1\ (\vec{x}) \\ \vdots \\ \partial g_4/\partial x_n\ (\vec{x}) \end{bmatrix} = \frac{1}{\sum_{i=1}^n e^{x_i}} \begin{bmatrix} e^{x_1} \\ \vdots \\ e^{x_n} \end{bmatrix}. \tag{67}$$

The Hessian is computed using more component-wise derivatives and the scalar quotient rule:

$$\frac{\partial^2 g_4}{\partial x_j\, \partial x_k}(\vec{x}) = \frac{\partial}{\partial x_k} \left(\frac{\partial g_4}{\partial x_j} \right) (\vec{x}) \tag{68}$$

$$= \frac{\partial}{\partial x_k} \frac{e^{x_j}}{\sum_{i=1}^n e^{x_i}} \tag{69}$$

$$= \frac{(\sum_{i=1}^n e^{x_i}) \frac{\partial}{\partial x_k} e^{x_j} - e^{x_j} \frac{\partial}{\partial x_k} (\sum_{i=1}^n e^{x_i})}{(\sum_{i=1}^n e^{x_i})^2} \tag{70}$$

$$= \frac{(\sum_{i=1}^n e^{x_i}) \frac{\partial}{\partial x_k} e^{x_j} - e^{x_j} e^{x_k}}{(\sum_{i=1}^n e^{x_i})^2} \tag{71}$$

$$= -\frac{e^{x_j + x_k}}{(\sum_{i=1}^n e^{x_i})^2} + \begin{cases} \frac{e^{x_k}}{\sum_{i=1}^n e^{x_i}}, & \text{if } j = k \\ 0, & \text{if } j \neq k. \end{cases} \tag{72}$$

This is the (i, j)th coordinate of the Hessian, whose entries can be defined as

$$[\nabla^2 g_4(\vec{x})]_{jk} = -\frac{e^{x_j + x_k}}{(\sum_{i=1}^n e^{x_i})^2} \qquad \forall j \neq k \tag{73}$$

$$\text{and} \qquad [\nabla^2 g_4(\vec{x})]_{jj} = \frac{e^{x_k}}{\sum_{i=1}^n e^{x_i}} - \frac{e^{x_j + x_k}}{(\sum_{i=1}^n e^{x_i})^2} \qquad \forall j. \tag{74}$$

For the purpose of self-grades, the above solution is fine, but we we can also write the Hessian as

$$\nabla^2 g_4(\vec{x}) = \operatorname{diag}(\nabla g_4(\vec{x})) - [\nabla g_4(\vec{x})][\nabla g_4(\vec{x})]^\top \tag{75}$$

where $\operatorname{diag}(\cdot)\colon \mathbb{R}^n \to \mathbb{R}^{n \times n}$ forms a diagonal matrix whose diagonal entries are the entries of the input vector, i.e.,

$$\operatorname{diag}(\vec{v}) = \begin{bmatrix} v_1 & & \\ & \ddots & \\ & & v_n \end{bmatrix}. \tag{76}$$

v. $g_5(\vec{x}) = g_4(A\vec{x} - \vec{b}) = \log(\sum_{i=1}^n e^{\vec{a}_i^\top \vec{x} - b_i})$. (Use the chain rule and the Jacobians computed in part 1(a); you can use the gradient ∇g_4 and Hessian $\nabla^2 g_4$ in your answer without having to rewrite it.)

Solution: As prompted, we use the chain rule to compute the gradient:

$$\nabla g_5(\vec{x}) = (Dg_5(\vec{x}))^\top \tag{77}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

$$= ([Dg_4(A\vec{x} - \vec{b})][D(A\vec{x} - \vec{b})])^\top \tag{78}$$
$$= ([Dg_4(A\vec{x} - \vec{b})]A)^\top \tag{79}$$
$$= A^\top [Dg_4(A\vec{x} - \vec{b})]^\top \tag{80}$$
$$= A^\top [\nabla g_4(A\vec{x} - \vec{b})]. \tag{81}$$

To compute the Hessian, we write

$$\nabla^2 g_5(\vec{x}) = D(\nabla g_5)(\vec{x}) \tag{82}$$
$$= D(A^\top [\nabla g_4(A\vec{x} - \vec{b})]). \tag{83}$$

To evaluate this product, we look component-by-component and use a multivariate chain rule:

$$[D(A^\top [\nabla g_4(A\vec{x} - \vec{b})])]_{jk} = \frac{\partial}{\partial x_k}(A^\top [\nabla g_4(A\vec{x} - \vec{b})])_j \tag{84}$$
$$= \frac{\partial}{\partial x_k} \sum_{i=1}^n a_{ij} [\nabla g_4(A\vec{x} - \vec{b})]_i \tag{85}$$
$$= \sum_{i=1}^n a_{ij} \frac{\partial}{\partial x_k} [\nabla g_4(A\vec{x} - \vec{b})]_i \tag{86}$$
$$= \sum_{i=1}^n a_{ij} \sum_{\ell=1}^n \frac{\partial [\nabla g_4(A\vec{x} - \vec{b})]_i}{\partial (A\vec{x} - \vec{b})_\ell} \frac{\partial (A\vec{x} - \vec{b})_\ell}{\partial x_k} \tag{87}$$
$$= \sum_{i=1}^n a_{ij} \sum_{\ell=1}^n \frac{\partial^2 g_4(A\vec{x} - \vec{b})}{\partial (A\vec{x} - \vec{b})_i \, \partial (A\vec{x} - \vec{b})_\ell} \frac{\partial (A\vec{x} - \vec{b})_\ell}{\partial x_k} \tag{88}$$
$$= \sum_{i=1}^n a_{ij} \sum_{\ell=1}^n [\nabla^2 g_4(A\vec{x} - \vec{b})]_{i\ell} [D(A\vec{x} - \vec{b})]_{\ell k} \tag{89}$$
$$= \sum_{i=1}^n a_{ij} \sum_{\ell=1}^n [\nabla^2 g_4(A\vec{x} - \vec{b})]_{i\ell} [A]_{\ell k} \tag{90}$$
$$= \sum_{i=1}^n a_{ij} ([\nabla^2 g_4(A\vec{x} - \vec{b})]A)_{ik} \tag{91}$$
$$= \sum_{i=1}^n [A^\top]_{ji} ([\nabla^2 g_4(A\vec{x} - \vec{b})]A)_{ik} \tag{92}$$
$$= (A^\top [\nabla^2 g_4(A\vec{x} - \vec{b})]A)_{jk}. \tag{93}$$

Thus,

$$\nabla^2 g_5(\vec{x}) = A^\top [\nabla^2 g_4(A\vec{x} - \vec{b})]A. \tag{94}$$

vi. $g_6(\vec{x}) = e^{\|\vec{x}\|_2^2} = e^{g_2(\vec{x})}$. (Use the chain rule and the Jacobians computed in part 1(a).)

Solution: We use the multivariable chain rule, obtaining

$$\nabla g_6(\vec{x}) = (Dg_6(\vec{x}))^\top \tag{95}$$
$$= ([D\exp(g_2(\vec{x}))][Dg_2(\vec{x})])^\top \tag{96}$$
$$= [Dg_2(\vec{x})]^\top [D\exp(g_2(\vec{x}))]^\top \tag{97}$$
$$= [\nabla g_2(\vec{x})][e^{g_2(\vec{x})}]^\top \tag{98}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

$$= [2\vec{x}][e^{g_2(\vec{x})}] \tag{99}$$
$$= 2e^{\|\vec{x}\|_2^2}\vec{x}, \tag{100}$$

since the transpose of a scalar is a scalar. For the Hessian, we obtain

$$\nabla^2 g_6(\vec{x}) = D(\nabla g_6)(\vec{x}) \tag{101}$$
$$= D(2e^{\|\vec{x}\|_2^2}\vec{x}). \tag{102}$$

Notice that the function being differentiated is of the form $f(\vec{x})\vec{x}$, for $f(\vec{x}) = 2e^{\|\vec{x}\|_2^2}$. Thus we obtain that

$$\nabla^2 g_6(\vec{x}) = D(f(\vec{x})\vec{x}) \tag{103}$$
$$= \vec{x}[\nabla f(\vec{x})]^\top + f(\vec{x})I. \tag{104}$$

It just remains to compute $\nabla f(\vec{x})$, which we compute componentwise using the scalar chain rule, and obtain

$$[\nabla f(\vec{x})]_j = \frac{\partial f}{\partial x_j}(\vec{x}) \tag{105}$$
$$= \frac{\partial}{\partial x_j}(2e^{\|\vec{x}\|_2^2}) \tag{106}$$
$$= 2e^{\|\vec{x}\|_2^2}\frac{\partial \|\vec{x}\|_2^2}{\partial x_j} \tag{107}$$
$$= 2e^{\|\vec{x}\|_2^2}[\nabla \|\vec{x}\|_2^2]_j \tag{108}$$
$$= 2e^{\|\vec{x}\|_2^2}[2\vec{x}]_j \tag{109}$$
$$= 4x_j e^{\|\vec{x}\|_2^2}. \tag{110}$$

Thus we have

$$\nabla f(\vec{x}) = 4e^{\|\vec{x}\|_2^2}\vec{x}. \tag{111}$$

This gives

$$\nabla^2 g_6(\vec{x}) = 4e^{\|\vec{x}\|_2^2}\vec{x}\vec{x}^\top + 2e^{\|\vec{x}\|_2^2}I \tag{112}$$
$$= 2e^{\|\vec{x}\|_2^2}(I + 2\vec{x}\vec{x}^\top). \tag{113}$$

vii. $g_7(\vec{x}) = e^{\|A\vec{x}-\vec{b}\|_2^2} = g_6(A\vec{x} - \vec{b})$. (Use the chain rule and the Jacobians computed in part 1(a); you can use the gradient ∇g_6 and Hessian $\nabla^2 g_6$ in your answer without having to rewrite it.)
Solution: Notice that in part 1(1(b))v, we did not use any specific functional properties of g_4, ∇g_4, or $\nabla^2 g_5$ to determine ∇g_5 and $\nabla^2 g_5$. Using the exact same analysis, we can conclude that

$$\nabla g_7 = A^\top[\nabla g_6(A\vec{x} - \vec{b})], \qquad \text{and} \qquad \nabla^2 g_7 = A^\top[\nabla^2 g_6(A\vec{x} - \vec{b})]A. \tag{114}$$

Consider the case now where all vectors and matrices above are scalar; do your answers above make sense? (No need to answer this in your submission.)

(c) Plot/hand-draw the level sets of the following functions:

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

i. $g(x_1, x_2) = \frac{x_1^2}{4} + \frac{x_2^2}{9}$
ii. $g(x_1, x_2) = x_1 x_2$

Also point out the gradient directions in the level-set diagram. Additionally, compute the first and second order Taylor series approximation around the point $(1, 1)$ for each function and comment on how accurately they approximate the true function.

Solution: Figures 1 and 2 contain the level sets and gradient directions for the given functions.

i. We first compute the first and second order partial derivatives of g as follows:

$$\frac{\partial g}{\partial x_1}(x_1, x_2) = \frac{x_1}{2}, \quad \frac{\partial g}{\partial x_2}(x_1, x_2) = \frac{2x_2}{9}, \tag{115}$$

$$\frac{\partial^2 g}{\partial x_1^2}(x_1, x_2) = \frac{1}{2}, \quad \frac{\partial^2 g}{\partial x_2 x_1}(x_1, x_2) = 0, \tag{116}$$

$$\frac{\partial^2 g}{\partial x_2^2}(x_1, x_2) = \frac{2}{9}, \quad \frac{\partial^2 g}{\partial x_1 x_2}(x_1, x_2) = 0. \tag{117}$$

The gradient of g is then given by,

$$\nabla g(x_1, x_2) = \begin{bmatrix} \frac{\partial g}{\partial x_1}(1, 1) \\ \frac{\partial g}{\partial x_2}(1, 1) \end{bmatrix}, \tag{118}$$

and the Hessian matrix is given by,

$$H(x_1, x_2) = \begin{bmatrix} \frac{\partial^2 g}{\partial x_1^2}(x_1, x_2) & \frac{\partial^2 g}{\partial x_1 x_2}(x_1, x_2) \\ \frac{\partial^2 g}{\partial x_2 x_1}(x_1, x_2) & \frac{\partial^2 g}{\partial x_2^2}(x_1, x_2) \end{bmatrix}. \tag{119}$$

The first order Taylor series approximation around $(1, 1)$ can be computed as:

$$g(x_1, x_2) \approx g(1, 1) + (\nabla g(1, 1))^\top \begin{bmatrix} x_1 - 1 \\ x_2 - 1 \end{bmatrix} \tag{120}$$

$$= \frac{13}{36} + \frac{x_1}{2} - \frac{1}{2} + \frac{2x_2}{9} - \frac{2}{9} = \frac{x_1}{2} + \frac{2x_2}{9} - \frac{13}{36}. \tag{121}$$

The second order Taylor series approximation around $(1, 1)$ can be computed as:

$$g(x_1, x_2) \approx g(1, 1) + (\nabla g(1, 1))^\top \begin{bmatrix} x_1 - 1 \\ x_2 - 1 \end{bmatrix} + \frac{1}{2} \begin{bmatrix} x_1 - 1 & x_2 - 1 \end{bmatrix} H(1, 1) \begin{bmatrix} x_1 - 1 \\ x_2 - 1 \end{bmatrix} \tag{122}$$

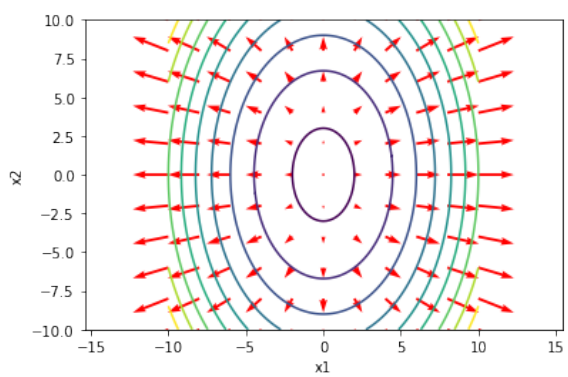

Figure 1: Level sets and gradient directions for the function $g(x_1, x_2) = \frac{x_1^2}{4} + \frac{x_2^2}{9}$.

 All Rights Reserved. This may not be publicly shared without explicit permission.

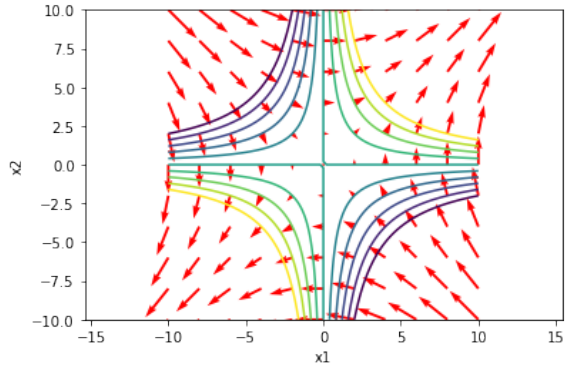

Figure 2: Level sets and gradient directions for the function $g(x_1, x_2) = x_1 x_2$.

$$= \frac{x_1}{2} + \frac{2x_2}{9} - \frac{13}{36} + \frac{1}{2}\left(\frac{1}{2}(x_1-1)^2 + \frac{2}{9}(x_2-1)^2\right) \tag{123}$$

$$= \frac{(x_1-1)^2}{4} + \frac{(x_2-1)^2}{9} + \frac{x_1}{2} + \frac{2x_2}{9} - \frac{13}{36}. \tag{124}$$

$$= \frac{x_1^2}{4} + \frac{x_2^2}{9} \tag{125}$$

The original function at (1.1,1.1) takes on the value 0.437. The first order approximation returns, evaluated at (1.1,1.1): $\frac{1.1}{2} + \frac{2.2}{9} - \frac{13}{36} = 0.433$. Additionally, observe that the second order approximation simplifies to return the original function!

ii. We follow the same steps as in the previous part of the problem. The partial derivatives for this g are given by:

$$\frac{\partial g}{\partial x_1}(x_1, x_2) = x_2, \quad \frac{\partial g}{\partial x_2}(x_1, x_2) = x_1, \tag{126}$$

$$\frac{\partial^2 g}{\partial x_1^2}(x_1, x_2) = 0, \quad \frac{\partial^2 g}{\partial x_2 x_1}(x_1, x_2) = 1, \tag{127}$$

$$\frac{\partial^2 g}{\partial x_2^2}(x_1, x_2) = 0, \quad \frac{\partial^2 g}{\partial x_1 x_2}(x_1, x_2) = 1. \tag{128}$$

The first order Taylor series approximation around $(1, 1)$ can be computed as:

$$g(x_1, x_2) \approx g(1,1) + (\nabla g(1,1))^\top \begin{bmatrix} x_1 - 1 \\ x_2 - 1 \end{bmatrix} \tag{129}$$

$$= 1 + x_1 - 1 + x_2 - 1 = x_1 + x_2 - 1. \tag{130}$$

The second order Taylor series approximation around $(1, 1)$ can be computed as:

$$g(x_1, x_2) \approx g(1,1) + (\nabla g(1,1))^\top \begin{bmatrix} x_1 - 1 \\ x_2 - 1 \end{bmatrix} + \frac{1}{2}\begin{bmatrix} x_1 - 1 & x_2 - 1 \end{bmatrix} H(1,1) \begin{bmatrix} x_1 - 1 \\ x_2 - 1 \end{bmatrix} \tag{131}$$

$$= x_1 + x_2 - 1 + \frac{1}{2}\left(2(x_1-1)(x_2-1)\right) \tag{132}$$

$$= (x_1-1)(x_2-1) + x_1 + x_2 - 1. \tag{133}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

$$= x_1 x_2 \tag{134}$$

The original function evaluated at (1.1,1.1) is 1.21. The first order approximation around $(1.1, 1.1)$ is 1.2, but the second order approximation again exactly represents the function!

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

2. Neural Networks and Backpropagation

Neural networks are parametric functions that have been widely used to fit complex patterns in vision and natural languages. Given some training data of the form $(\vec{x}_i, y_i)$, a neural network $\mathcal{N}$ is trained to minimize a *loss function* on the data. This is often done using *gradient descent*, an optimization method we will cover later in this class. Gradient descent requires us to compute the gradients of the loss function with respect to the parameters of the neural network. In practice, computational frameworks for neural networks compute the gradients automatically and efficiently via *back-propagation*, which uses the chain rule to recursively compute the gradients of the loss function. In this problem, we study a toy neural network trained on a single data point $(\vec{x}, y)$.

In particular, consider the following simplified three-layer neural network $\mathcal{N}$, representing a map from $\mathbb{R}^d$ to $\mathbb{R}$ whose parameters are $(\vec{w}_1, w_2, w_3) \in \mathbb{R}^d \times \mathbb{R} \times \mathbb{R}$:

$$p_1 = \vec{w}_1^\top \vec{x} \tag{135}$$

$$h_1 = \sigma(p_1) \tag{136}$$

$$p_2 = w_2 h_1 \tag{137}$$

$$h_2 = \sigma(p_2) \tag{138}$$

$$z = w_3 h_2, \tag{139}$$

where $\sigma \colon \mathbb{R} \to \mathbb{R}$ is a nonlinear function (also called the "activation function"), whose derivative is denoted by $\sigma' \colon \mathbb{R} \to \mathbb{R}$.

We want the output of the network $z = \mathcal{N}(\vec{x})$ to match the true label y. A natural choice of the loss function encouraging this behavior is the squared loss:

$$L(y, z) \doteq \frac{1}{2}(y - z)^2. \tag{140}$$

In the parts that follow, we will compute the derivative of L with respect to the parameters $\vec{w}_1, w_2, w_3$.

(a) Compute the following gradients and partial derivatives sequentially from left-to-right:

$$\frac{\partial L}{\partial z}, \quad \frac{\partial L}{\partial w_3}, \quad \frac{\partial L}{\partial h_2}, \quad \frac{\partial L}{\partial p_2}, \quad \frac{\partial L}{\partial w_2}, \quad \frac{\partial L}{\partial h_1}, \quad \frac{\partial L}{\partial p_1}, \quad \nabla_{\vec{w}_1} L, \quad \nabla_{\vec{x}} L. \tag{141}$$

Here $\nabla_{\vec{x}} L$ is the gradient whose entries are the derivatives of L with respect to the entries of $\vec{x}$, etc. We compute the first 4 derivatives for you.

$$\frac{\partial L}{\partial z} = z - y, \quad \frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z} h_2, \quad \frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial z} w_3, \quad \frac{\partial L}{\partial p_2} = \frac{\partial L}{\partial h_2} \sigma'(p_2) \tag{142}$$

Note how $\frac{\partial L}{\partial w_3}$ can be calculated using $\frac{\partial L}{\partial z}$ and $\frac{\partial L}{\partial p_2}$ can be calculated using $\frac{\partial L}{\partial h_2}$. In numerical computation, the result of $\frac{\partial L}{\partial z}$ and $\frac{\partial L}{\partial h_2}$ can thus be reused. This technique of saving computations for calculating the derivatives of a neural network is called *back-propagation*.

Use the chain rule to calculate the 5 remaining derivatives $\frac{\partial L}{\partial w_2}$, $\frac{\partial L}{\partial h_1}$, $\frac{\partial L}{\partial p_1}$,$\nabla_{\vec{w}_1} L$, and $\nabla_{\vec{x}} L$.

Solution: We compute each term via chain rule.

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial p_2} h_1 \tag{143}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial p_2} w_2 \tag{144}$$

$$\frac{\partial L}{\partial p_1} = \frac{\partial L}{\partial h_1} \sigma'(p_1) \tag{145}$$

$$\nabla_{\vec{w}_1} L = \frac{\partial L}{\partial p_1} \vec{x} \tag{146}$$

$$\nabla_{\vec{x}} L = \frac{\partial L}{\partial p_1} \vec{w}_1 \tag{147}$$

(b) Now suppose that Equation (139) is written as

$$z' = w_3 h_2 + h_1. \tag{148}$$

That is, we define a new neural network $\mathcal{N}'$ with parameters $(\vec{w}_1, w_2, w_3) \in \mathbb{R}^d \times \mathbb{R} \times \mathbb{R}$ as follows.

$$p_1 = \vec{w}_1^\top \vec{x} \tag{149}$$

$$h_1 = \sigma(p_1) \tag{150}$$

$$p_2 = w_2 h_1 \tag{151}$$

$$h_2 = \sigma(p_2) \tag{152}$$

$$z' = w_3 h_2 + h_1. \tag{153}$$

This introduces a change in the network architecture, called the *skip connection*.

Again, compute the following gradients and partial derivatives with respect to the loss function $L(y, z') = \frac{1}{2}(y - z')^2$:

$$\frac{\partial L}{\partial z'}, \quad \frac{\partial L}{\partial w_3}, \quad \frac{\partial L}{\partial h_2}, \quad \frac{\partial L}{\partial p_2}, \quad \frac{\partial L}{\partial w_2}, \quad \frac{\partial L}{\partial h_1}, \quad \frac{\partial L}{\partial p_1}, \quad \nabla_{\vec{w}_1} L, \quad \nabla_{\vec{x}} L. \tag{154}$$

We compute the first 4 derivatives for you.

$$\frac{\partial L}{\partial z'} = z' - y, \quad \frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z'} h_2, \quad \frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial z'} w_3, \quad \frac{\partial L}{\partial p_2} = \frac{\partial L}{\partial h_2} \sigma'(p_2). \tag{155}$$

Use the chain rule to calculate the 5 remaining derivatives $\frac{\partial L}{\partial w_2}$, $\frac{\partial L}{\partial h_1}$, $\frac{\partial L}{\partial p_1}$,$\nabla_{\vec{w}_1} L$, and $\nabla_{\vec{x}} L$.

Solution:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial p_2} h_1 \tag{156}$$

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial z'} \frac{\partial z'}{\partial h_1} = \frac{\partial L}{\partial z'} \left(1 + w_3 \frac{\partial h_2}{\partial h_1}\right) = \frac{\partial L}{\partial z'} \left(1 + w_3 \frac{\partial h_2}{\partial p_2} \frac{\partial p_2}{\partial h_1}\right) = \frac{\partial L}{\partial z'} (1 + w_3 \sigma'(p_2) w_2) \tag{157}$$

$$\frac{\partial L}{\partial p_1} = \frac{\partial L}{\partial z'} \frac{\partial z'}{\partial p_1} = \frac{\partial L}{\partial z'} \left(w_3 \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial p_1} + \frac{\partial h_1}{\partial p_1}\right) = \frac{\partial L}{\partial z'} (w_3 \sigma'(p_2) w_2 \sigma'(p_1) + \sigma'(p_1)) \tag{158}$$

$$\nabla_{\vec{w}_1} L = \frac{\partial L}{\partial p_1} \vec{x} \tag{159}$$

$$\nabla_{\vec{x}} L = \frac{\partial L}{\partial p_1} \vec{w}_1. \tag{160}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

(c) In optimizing a neural network using gradient descent, we need the gradient of the loss function with respect to the parameters of the network. Please express $\frac{\partial L}{\partial w_3}$, $\frac{\partial L}{\partial w_2}$, and $\nabla_{\vec{w}_1} L$ for $\mathcal{N}$ and $\mathcal{N}'$ respectively with no dependence on partial derivatives of other variables. We compute $\frac{\partial L}{\partial w_3}$ for you, as follows.

- For $\mathcal{N}$, we have $\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z} h_2 = (z - y)h_2$.
- For $\mathcal{N}'$, we have $\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z'} h_2 = (z' - y)h_2$.

Express the remaining derivatives $\frac{\partial L}{\partial w_2}$ and $\nabla_{\vec{w}_1} L$ within $\mathcal{N}$ and $\mathcal{N}'$.

Solution: For $\mathcal{N}$, we have

$$\begin{aligned}
\frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial p_2} h_1 && (161)\\
&= \frac{\partial L}{\partial h_2} \sigma'(p_2) h_1 && (162)\\
&= \frac{\partial L}{\partial z} w_3 \sigma'(p_2) h_1 && (163)\\
&= (z - y) w_3 \sigma'(p_2) h_1 && (164)\\
\nabla_{\vec{w}_1} L &= \frac{\partial L}{\partial p_1} \vec{x} && (165)\\
&= \frac{\partial L}{\partial h_1} \sigma'(p_1) \vec{x} && (166)\\
&= \frac{\partial L}{\partial p_2} w_2 \sigma'(p_1) \vec{x} && (167)\\
&= \frac{\partial L}{\partial h_2} \sigma'(p_2) w_2 \sigma'(p_1) \vec{x} && (168)\\
&= \frac{\partial L}{\partial z} w_3 \sigma'(p_2) w_2 \sigma'(p_1) \vec{x} && (169)\\
&= (z - y) w_3 \sigma'(p_2) w_2 \sigma'(p_1) \vec{x}. && (170)
\end{aligned}$$

For $\mathcal{N}'$, we have

$$\begin{aligned}
\frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial p_2} h_1 && (171)\\
&= \frac{\partial L}{\partial h_2} \sigma'(p_2) h_1 && (172)\\
&= \frac{\partial L}{\partial z'} w_3 \sigma'(p_2) h_1 && (173)\\
&= (z' - y) w_3 \sigma'(p_2) h_1 && (174)\\
\nabla_{\vec{w}_1} L &= \frac{\partial L}{\partial p_1} \vec{x} && (175)\\
&= \frac{\partial L}{\partial z'} (w_3 \sigma'(p_2) w_2 \sigma'(p_1) + \sigma'(p_1)) \vec{x} && (176)\\
&= (z' - y)(w_3 \sigma'(p_2) w_2 \sigma'(p_1) + \sigma'(p_1)) \vec{x} && (177)\\
&= (z' - y)(w_3 \sigma'(p_2) w_2 + 1) \sigma'(p_1) \vec{x}. && (178)
\end{aligned}$$

(d) Many activation functions σ have the property that $\sigma' \leq 1$. For example, the sigmoid function $\sigma(p) = \frac{1}{1+e^{-p}}$ is sometimes used as an activation function. Its derivative, $\sigma'(p) = \frac{e^{-p}}{(1+e^{-p})^2}$ has the range $(0, 1/4]$.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

That is, $\sigma' < 1$. Consider the case when σ' is much smaller than 1, such that $\sigma'(p)\sigma'(q) \approx 0$ for any p, q, but $\sigma'(p) \not\approx 0$ for any p. Consider the derivatives $\frac{\partial L}{\partial w_3}$, $\frac{\partial L}{\partial w_2}$, and $\nabla_{\vec{w}_1} L$ within the neural network $\mathcal{N}$; with the above approximations, which of them will approximately be zero? Also answer this question for the neural network $\mathcal{N}'$.

NOTE: Some of the above gradients will indeed be approximately zero, and this is called the *vanishing gradient* problem in deep learning.

Solution: Yes, $\nabla_{\vec{w}_1} L$ for $\mathcal{N}$ involves a product of σ's and thus approximately vanishes. But all other derivatives don't go to zero for both $\mathcal{N}$ and $\mathcal{N}'$, including $\nabla_{\vec{w}_1} L$ for $\mathcal{N}'$.

Notably, $\nabla_{\vec{w}_1} L$ does not go to zero for $\mathcal{N}'$ because the network architecture of $\mathcal{N}'$ includes a skip connection that adds h_1 to the last layer z', adding a $+1$ in the gradient expression.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

3. PCA and Senate Voting Data

In this problem, we consider a matrix of senate voting data, which we manipulate in Python. The data is contained in a $n \times d$ data matrix X, where each row corresponds to a senator and each column to a bill. Each entry of X is either $1, -1$ or 0, depending on whether the senator voted for the bill, against the bill, or abstained, respectively. Please compute your answers using the Google Colab `senator_pca.ipynb`. The sub-parts of this problem can be answered in the notebook itself in the space provided and can be submitted as an attachment to this PDF using the "File > Print > Download as PDF" feature that Google Colab supports. To use Google Colab, you need to make your own private copy of the starter code. Please follow these steps:

Open the Link: Click this link. This will open the Google Colab notebook in "read-only" mode.

Make Your Own Copy: Do not try to type in the notebook yet. Instead, go to the top menu and click File > Save a copy in Drive.

Start Coding: A new tab will open with your personal copy of the notebook and you can safely begin writing your code.

(a) Suppose we want to assign a *score* to each senator based on their voting pattern, and then observe the empirical variance of these scores. To describe this, let us choose a $\vec{a} \in \mathbb{R}^d$ and a scalar $b \in \mathbb{R}$. We define the score for senator i as:

$$f(\vec{x}_i, \vec{a}, b) = \vec{x}_i^\top \vec{a} + b, \quad i = 1, 2, \ldots, n. \tag{179}$$

Note that $\vec{x}_i^\top$ denotes the ith row of X and is a row vector of length d, as in the previous problem.

Let us denote by $\vec{z} = f(X, \vec{a}, b)$ the column vector of length n obtained by stacking the scores for each senator. Then

$$\vec{z} = f(X, \vec{a}, b) = X\vec{a} + b\vec{1} \in \mathbb{R}^n \tag{180}$$

where $\vec{1}$ is a vector with all entries equal to 1. Let us denote the mean value of $\vec{z}$ by $\mu(\vec{z}) = \frac{1}{n}\vec{1}^\top \vec{z}$. Let $\vec{\mu}(X) \in \mathbb{R}^d$ denote the vector containing the mean of each column of X. Then

$$\begin{aligned}
\mu(\vec{z}) &= \frac{1}{n}\sum_{i=1}^{n} z_i && (181)\\
&= \frac{1}{n}\sum_{i=1}^{n} \left(\vec{a}^\top \vec{x}_i + b\right) && (182)\\
&= \vec{a}^\top \left(\frac{1}{n}\sum_{i=1}^{n} \vec{x}_i\right) + b && (183)\\
&= \vec{a}^\top \vec{\mu}(X) + b && (184)
\end{aligned}$$

The empirical variance of the scores can then be obtained as

$$\begin{aligned}
\sigma^2(\vec{z}) &= \frac{1}{n}(\vec{z} - \mu(\vec{z})\vec{1})^\top(\vec{z} - \mu(\vec{z})\vec{1}) && (185)\\
&= \frac{1}{n}((X\vec{a} + b\vec{1}) - (\vec{a}^\top\vec{\mu}(X) + b)\vec{1})^\top((X\vec{a} + b\vec{1}) - (\vec{a}^\top\vec{\mu}(X) + b)\vec{1}) && (186)\\
&= \frac{1}{n}(X\vec{a} + b\vec{1} - (\vec{a}^\top\vec{\mu}(X))\vec{1} - b\vec{1})^\top(X\vec{a} + b\vec{1} - (\vec{a}^\top\vec{\mu}(X))\vec{1} - b\vec{1}) && (187)\\
&= \frac{1}{n}(X\vec{a} - (\vec{a}^\top\vec{\mu}(X))\vec{1})^\top(X\vec{a} - (\vec{a}^\top\vec{\mu}(X))\vec{1}) && (188)\\
&= \frac{1}{n}(X\vec{a} - \vec{1}\vec{\mu}(X)^\top\vec{a})^\top(X\vec{a} - \vec{1}\vec{\mu}(X)^\top\vec{a}) && (189)
\end{aligned}$$

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

$$= \frac{1}{n}((X - \vec{1}\vec{\mu}(X)^\top)\vec{a})^\top((X - \vec{1}\vec{\mu}(X)^\top)\vec{a}) \tag{190}$$

$$= \frac{1}{n}\vec{a}^\top(X - \vec{1}\vec{\mu}(X)^\top)^\top(X - \vec{1}\vec{\mu}(X)^\top)\vec{a}. \tag{191}$$

Note that this variance is therefore a function of the "centered" data matrix $X - \vec{1}\vec{\mu}(X)^\top$ in which the mean of each column is zero. It also does not depend on b.

For the remainder of this problem, we assume that the data has been pre-centered (i.e., $\vec{\mu}(X) = \vec{0}$); note that this has been pre-computed for you in the code notebook. Assume also that $b = 0$, so that $\mu(\vec{z}) = 0$. Defining $f(X, \vec{a}) \doteq f(X, \vec{a}, 0)$ and replacing $\vec{z}$ with $f(X, \vec{a})$, we can then write the simpler empirical variance formula

$$\sigma^2(f(X, \vec{a})) = \frac{1}{n}\vec{a}^\top X^\top X\vec{a}. \tag{192}$$

Suppose we restrict $\vec{a}$ to have unit-norm. In the provided code, find the maximum empirical variance $\sigma^2(f(X, \vec{a}))$ over all unit-norm $\vec{a}$, and find the $\vec{a}$ that maximizes it.

Solution: The vector $\vec{a}$ can be computed as the first principal component of the data matrix X. For the full solution, see the notebook. (Vector $\vec{a}$ is the 542×1 column vector `a_1`, and the maximum variance is just under 150.)

As a side note, computing PCA by finding the direction of maximum variance is only one possible way of conceptualizing the computation; it's also equivalent to minimixing the reconstruction error in a lower-dimensional space. We'll explore this equivalence further in a future homework!

(b) We next consider party affiliation as a predictor for how a senator will vote. Follow the instructions in the notebook to compute the mean voting vector for each party and relate it to the direction of maximum variance.

Solution: For solution, see Google Colab link.

(c) Recall from problem 1 that given a vector $\vec{z} = X\vec{u}$ (i.e., the vector of scalar projections of each row of X along $\vec{u}$), we can compute its empirical variance as

$$\sigma^2(\vec{z}) = \vec{u}^\top\Sigma\vec{u}, \tag{193}$$

where $\Sigma(X) = \frac{X^\top X}{n}$ is the empirical covariance matrix of X. We will show in a future homework problem that the variance along each principal component $\vec{a}_i$ is precisely its corresponding eigenvalue of $\Sigma(X)$, i.e., $\lambda_i\{\Sigma(X)\}$. (For now, just note that this fact should make intuitive sense, since PCA is searching for directions of maximum variance of the data, and these occur along the covariance matrix's eigenvectors.) In the Notebook, compute the sum of the variance along $\vec{a}_1$ and $\vec{a}_2$ and plot the data projected on the $\vec{a}_1$–$\vec{a}_2$ plane.

Solution: For solution, see Google Colab link.

(d) Suppose we want to find the bills that are most and least contentious — i.e., those that have high variability in senators' votes, and those for which voting was almost unanimous. Follow the instructions in the notebook to compute a measure of "contentiousness" for each bill, plot the vote counts for exemplar bills, and comment on the voting trends.

Solution: For solution, see Google Colab link.

(e) Suppose we want to infer the political affiliations of two senators whose voting records are known to us. Follow the instructions in the notebook to infer the political affiliation of the Green and Grey colored senators using PCA.

Solution: For solution, see Google Colab link.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

(f) Finally, we can use the defined score $f(X, \vec{a}, b)$, computed along the first principal component $\vec{a}_1$ to classify the most and least "extreme" senators based on their voting record. Follow the instructions in the Notebook to compute these scores and comment on their relationship to partisan affiliation.

Solution: For solution, see Google Colab link.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

4. SVD Transformation

In this problem we will interpret the linear map defined by matrix $A \in \mathbb{R}^{n \times n}$ by looking at its singular value decomposition, $A = UDV^\top$. Recall that here $U, D, V \in \mathbb{R}^{n \times n}$ and U, V are orthonormal matrices while D is a diagonal matrix. We will first look at how $V^\top, D$ and U each separately transform the unit circle $C = \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 \leq 1\}$ and then look at their effect as a whole. This problem has an associated Google Colab, `svd_transformation.ipynb` that contains parts (b, c, d, e) of the problem. These sub-parts can be answered in the space provided in the notebook itself and can be submitted as an attachment to the solution PDF using the "File > Print > Download as PDF" feature that Google Colab supports. To use Google Colab, you need to make your own private copy of the starter code. Please follow these steps:

Open the Link: Click this link. This will open the Google Colab notebook in "read-only" mode.

Make Your Own Copy: Do not try to type in the notebook yet. Instead, go to the top menu and click File > Save a copy in Drive.

Start Coding: A new tab will open with your personal copy of the notebook and you can safely begin writing your code.

(a) Let $\vec{x} \in \mathbb{R}^n$ and $\vec{z} = V^\top \vec{x}$. Show that $\vec{x} = \sum_{i=1}^n z_i \vec{v}_i$, where $\vec{z} = \begin{bmatrix} z_1 & \cdots & z_n \end{bmatrix}^\top$ and $V = \begin{bmatrix} \vec{v}_1 & \cdots & \vec{v}_n \end{bmatrix}$. This shows that $\vec{z}$ is the vector of coordinates that represents $\vec{x}$ in the basis defined by the columns of V.

Solution: Since the columns of V form a basis for $\mathbb{R}^n$, we can represent $\vec{x} \in \mathbb{R}^n$ as $V\vec{z}$ for some $\vec{z}$ in $\mathbb{R}^n$. Then,

$$V^\top \vec{x} = V^\top V \vec{z} \tag{194}$$

$$= \vec{z}. \tag{195}$$

The last equality follows since V is an orthonormal matrix.

For the rest of the problem we restrict ourselves to the case where $A \in \mathbb{R}^{2 \times 2}$ and move to the notebook.

Solution: The solutions for the rest of the parts can be found in the Google Colab link.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.

5. Homework Process

With whom did you work on this homework? List the names and SIDs of your group members.

NOTE: If you didn't work with anyone, you can put "none" as your answer.

© UCB EECS 127/227AT, Spring 2026. All Rights Reserved. This may not be publicly shared without explicit permission.